

บทที่ 7

หลักการพัฒนาโปรแกรมด้วยวิธีการเชิงวัตถุ (Object-Oriented Programming)

เทคโนโลยีในการพัฒนาโปรแกรมเชิงวัตถุเป็นหลักการทางวิศวกรรม ซึ่งบรรจุไว้ด้วยองค์ประกอบต่างๆ ที่เรียกว่าโมเดลของวัตถุ (Object Model) ในโมเดลเชิงวัตถุนั้นแบ่งออกเป็นการแยกแยะเอกลักษณ์ (Abstraction) การซ่อนรายละเอียด (Encapsulation) การรวมกลุ่มความสัมพันธ์ (Modularity) ลำดับชั้น (Hierarchy) ชนิด (Typing) การทำงานพร้อมกัน (Concurrency) และการรักษาสถานะ (Persistence) สำหรับลักษณะการวิเคราะห์และออกแบบระบบด้วยวิธีการเชิงวัตถุนั้น จะแตกต่างจากการออกแบบระบบด้วยวิธีโครงสร้างเนื่องจากรูปแบบในการคิดและการมองระบบจะต่างกัน นอกจากนี้การออกแบบและพัฒนาระบบแบบโครงสร้างจะขึ้นอยู่กับภาษาที่ใช้ ส่วนการวิเคราะห์และออกแบบระบบจะขึ้นอยู่กับภาษาในการเขียนโปรแกรมเชิงวัตถุเช่นกัน และในภาษาสำหรับการเขียนโปรแกรมเชิงวัตถุแต่ละภาษาก็มีรายละเอียดปลีกย่อยที่ต่างกันอีกด้วย

7.1 หลักการเชิงวัตถุ (Object-Oriented)

การพัฒนาโปรแกรมแบบโครงสร้างนั้น จะพยายามให้นักพัฒนาระบบแก้ปัญหาด้วยการแบ่งปัญหาวางออกเป็นส่วนๆ แล้วแก้ปัญหาในแต่ละส่วนด้วยอัลกอริทึม (Algorithm) และ โครงสร้างข้อมูล (Data Structure) เฉพาะ สำหรับการพัฒนาระบบด้วยวิธีเชิงวัตถุก็เช่นเดียวกัน แต่จะมีมุมมองต่อปัญหาเป็นวัตถุประกอบกับการเขียนโปรแกรมเชิงวัตถุ การพัฒนาระบบเชิงวัตถุไม่เพียงจะเป็นแนวความคิดใหม่ในการเขียนโปรแกรมเท่านั้น แต่ยังมีข้อดีจากโมเดลของวัตถุ ซึ่งเป็นแนวความคิดใหม่ในวงการคอมพิวเตอร์อีกด้วย สามารถนำหลักการพัฒนาโปรแกรมเชิงวัตถุไปประยุกต์ใช้กับการออกแบบได้ทั้งในส่วนติดต่อกับผู้ใช้ (User Interface) ระบบฐานข้อมูลหรือแม้แต่การออกแบบสถาปัตยกรรมคอมพิวเตอร์ เนื่องจากหลักการของการพัฒนาโปรแกรมนั้นเป็นแนวความคิดที่จัดการกับความซับซ้อนของสิ่งต่างๆ อย่างมีระบบ ซึ่งสามารถนำไปประยุกต์ใช้ในงานต่างๆ ได้อย่างกว้างขวาง

การวิเคราะห์และออกแบบระบบด้วยวิธีเชิงวัตถุ เป็นวิวัฒนาการมิใช่เป็นการปฏิวัติการวิเคราะห์และออกแบบระบบที่มีอยู่ เนื่องจากการพัฒนาระบบด้วยวิธีเชิงวัตถุนั้นยังคงข้อดีในการพัฒนาระบบแบบเก่าแต่ก็ได้เพิ่มข้อดีใหม่ๆ เข้าไปด้วย ในการพัฒนาระบบแบบเก่าจะพบกับความซับซ้อน เนื่องจากการแบ่งระบบออกเป็นส่วนใหญ่ๆ เมื่อส่วนย่อยเหล่านี้มีมากขึ้นส่งผลให้การดูแลและแก้ไขระบบเป็นไปได้ยาก ในขณะที่หลักการของวัตถุนั้น มีการแบ่งปัญหาวางออกเป็นวัตถุ ซึ่งมีความผูกพันกันน้อย (Loose Couple) ทำให้การดูแลและแก้ไขส่วนต่างๆ ของระบบหรือวัตถุสามารถทำได้ง่าย และมีผลกระทบต่อส่วนอื่นๆ ของระบบน้อยที่สุด

แนวความคิดของการพัฒนาระบบด้วยวิธีเชิงวัตถุเกิดขึ้นครั้งแรกเมื่อประมาณ 20 ปีที่ผ่านมา โดยเริ่มจากการออกแบบสถาปัตยกรรมฮาร์ดแวร์ของคอมพิวเตอร์ นักพัฒนาฮาร์ดแวร์ได้ใช้แนวความคิดของการพัฒนาระบบด้วยวิธีเชิงวัตถุช่วยในการลดช่องว่างระหว่างการแยกแยะเอกลักษณ์ในระดับสูง เช่น การเขียนโปรแกรม และการแยกแยะเอกลักษณ์ในระดับล่างซึ่งก็คือการทำงานของตัวฮาร์ดแวร์ ด้วยการนำหลักการ

พัฒนาระบบด้วยวิธีเชิงวัตถุมาใช้ ทำให้การออกแบบและการพัฒนาฮาร์ดแวร์ สามารถตรวจสอบความผิดพลาดได้ง่าย ช่วยเพิ่มประสิทธิภาพในการปฏิบัติคำสั่ง ลักษณะของคำสั่งของฮาร์ดแวร์ มีการคอมไพล์จากภาษาระดับสูงเป็นภาษาระดับต่ำที่ไม่ซับซ้อน และยังช่วยลดพื้นที่ในการเก็บข้อมูลอีกด้วยตัวอย่างระบบคอมพิวเตอร์ที่ออกแบบโดยใช้แนวความคิดเชิงวัตถุ เช่น Burroughs 5000, Plessey 250 CAP, Intel 432 , IBM System/38 และ Rational R1000 เป็นต้น

7.2 คุณสมบัติของหลักการเชิงวัตถุ

7.2.1 Encapsulation/Information Hiding

Encapsulation/Information Hiding หมายถึง การแยกส่วนลักษณะภายนอก (Interface) ที่ติดต่อกับวัตถุอื่นๆ ออกจากส่วนที่ทำงาน (Implement) ภายในวัตถุซึ่งจะถูกใช้เฉพาะภายในวัตถุซึ่งมีข้อดีคือเพิ่มความสามารถในการปกป้องข้อมูลก่อนได้รับอนุญาต ทั้งนี้ตัววัตถุต้องสามารถทราบและทำการเปลี่ยนค่าได้ด้วยตัวเองซึ่งมีผลทำให้การควบคุมดูแลวัตถุ (Object Maintenance) ง่ายขึ้น

7.2.2 Inheritance

การถ่ายทอดคุณสมบัติ (Inheritance) คือ ลักษณะความสัมพันธ์ของคลาสที่ใช้ข้อมูลและพฤติกรรมร่วมกันโดยมีความสัมพันธ์ระหว่างคลาสในลักษณะ “is-a Relationship” ซึ่งมีหลักการในการถ่ายทอดคุณสมบัติของคลาสอันประกอบด้วยส่วนประกอบ (Attribute) และฟังก์ชันการทำงาน (Method) จากคลาสที่มีลำดับสูงกว่า (Super Class) มายังคลาสที่มีลำดับต่ำกว่า (Subclass) ซึ่งมีประโยชน์ในด้านความสอดคล้อง (Consistency) คือเมื่อเปลี่ยน Attribute หรือ Method ที่ Super Class จะมีผลทำให้ค่าใน Subclass เปลี่ยนไปด้วย นอกจากนี้ยังมีประโยชน์ในการนำวัตถุมาใช้ใหม่ด้วย

7.2.3 Polymorphism

Polymorphism คือ คุณสมบัติของคลาสที่แบ่งแยกลักษณะการทำงานวัตถุ ให้มีความสามารถในการทำงานในหลายๆ รูปแบบ การส่ง message เดียวกันให้วัตถุที่ต่างกันทำให้วัตถุสามารถแสดงการทำงานที่ต่างกันออกมาได้ ข้อดีของ Polymorphism คือ การสนับสนุนการนำโปรแกรมกลับมาใช้ใหม่และสามารถเปลี่ยนแปลงซอฟต์แวร์ได้ในระหว่างการพัฒนา

7.2.4 Abstraction

Abstraction คือคุณสมบัติในการแยกเอกลักษณ์ซึ่งเป็นวิธีการที่ใช้ในการแบ่งแยกสิ่งต่างๆ เพื่อลดความซับซ้อนของสิ่งๆ นั้น โดยแยกเอาเฉพาะสิ่งที่สนใจออกมา

Abstraction เป็นการแสดงคุณลักษณะที่สำคัญของวัตถุ วัตถุแต่ละตัวมีลักษณะที่แตกต่างกัน เช่น รถยนต์หากมองในแง่ของผู้ขับขี่ก็หมายถึง ยานพาหนะที่สามารถพาผู้โดยสารไปยังจุดหมายปลายทางได้ แต่ในแง่ของช่างเครื่องยนต์ รถยนต์หมายถึงชุดของเครื่องยนต์และอุปกรณ์ต่างๆ ที่ทำงานร่วมกัน เป็นต้น

7.3 ภาษาในการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming : OOP)

การเขียนโปรแกรมเชิงวัตถุ เป็นวิธีการในการสร้างโปรแกรม โปรแกรมจะถูกจัดให้มีโครงสร้างในการทำงานร่วมกันของวัตถุ ซึ่งวัตถุนั้นจะเป็นตัวตนหรือเป็นอินสแตนซ์ (Instance) ของคลาส และคลาสก็เป็นส่วนหนึ่งของลำดับชั้น (Hierarchy) ของคลาสที่มีความสัมพันธ์ในการสืบทอดคุณสมบัติ (Inheritance) โดยภาษาที่จะเป็นการเขียนโปรแกรมเชิงวัตถุได้นั้นจะต้องมีคุณสมบัติสามข้อคือ

7.3.1 จะต้องเป็นภาษาที่ใช้วัตถุมีอัลกอริทึมในการแก้ปัญหาเป็นหลัก

ถ้าเป็นภาษาในการเขียนโปรแกรมแบบเก่า หากเขียนโปรแกรมติดต่อกับอุปกรณ์สื่อสารก็จะแบ่งข้อมูลส่วนต่างๆ ติดต่อกับโมเด็มตามฟังก์ชันการทำงานต่างๆ เช่น รับ ส่ง หรือตรวจสอบข้อมูล แต่ถ้าเป็นภาษาในการเขียนโปรแกรมเชิงวัตถุแล้วผู้เขียนโปรแกรมจะต้องมองโมเด็มให้เป็นวัตถุตัวหนึ่งโดยในวัตถุโมเด็มก็อาจจะประกอบด้วยวัตถุย่อยๆ และวัตถุย่อยๆ เหล่านี้ก็จะมีการสื่อสารซึ่งกันและกัน สรุปคือ โปรแกรมคู่มือที่วัตถุเป็นหลัก

7.3.2 วัตถุที่ปรากฏในการเขียนโปรแกรมต้องเป็นอินสแตนซ์ของคลาส

เนื่องจากว่าหลักการของการเขียนโปรแกรมเชิงวัตถุมองวัตถุเป็นหลักซึ่งวัตถุนี้ไม่สามารถกำหนดขึ้นมาลอยๆ ได้ จำเป็นต้องมีลักษณะคุณสมบัติของตัวเองซึ่งจะถูกกำหนดไว้ในคลาส ดังนั้นวัตถุจะต้องเป็นอินสแตนซ์ของคลาส

7.3.3 คลาสจะต้องมีความสามารถในการถ่ายทอดคุณสมบัติ

นอกเหนือจากหน้าที่ในการกำหนดคุณสมบัติของคลาส และมีตัวแทนของคลาสที่เรียกว่าอินสแตนซ์แล้ว แต่ละคลาสจำเป็นต้องมีความสัมพันธ์กันด้วย คุณสมบัติที่สำคัญต่อการกำหนดความสัมพันธ์ของคลาสก็คือคลาสต้องถ่ายทอดคุณสมบัติได้ หากขาดคุณสมบัติข้อนี้ไปก็ไม่มีประโยชน์อันใดในการพัฒนาระบบเชิงวัตถุขึ้นมา

ถ้าภาษาใดขาดคุณสมบัติหนึ่งในสามข้อนี้ไปก็จะไม่เรียกว่าเป็นภาษาการเขียนโปรแกรมเชิงวัตถุ ตัวอย่างเช่น การสร้างคลาสวัตถุ แต่ไม่สนับสนุนการสืบทอดคุณสมบัติของคลาส เราจะเรียกภาษานั้นว่า ภาษาที่ทำงานบน Abstract Data Type หรือเรียกว่าภาษาในการเขียนโปรแกรมบนพื้นฐานของวัตถุ (Object-Based) มิใช่ภาษาในการเขียนโปรแกรมเชิงวัตถุ (Object Oriented)

ตัวอย่างของภาษาในการเขียนโปรแกรมเชิงวัตถุ เช่น Smalltalk, Object Pascal, C++, Eiffel, CLOS และ Java ส่วนภาษาในการเขียนโปรแกรมบนพื้นฐานของวัตถุก็ เช่น Ada เป็นต้น ซึ่งในการที่จะเขียนโปรแกรมด้วยภาษาเชิงวัตถุให้ได้อย่างมีประสิทธิภาพ ผู้เขียนโปรแกรมจะต้องมีการออกแบบด้วยหลักการพัฒนาระบบด้วยวิธีเชิงวัตถุเช่นกัน

7.4 การออกแบบระบบด้วยวิธีเชิงวัตถุ (Object-Oriented Design : OOD)

ก่อนที่จะลงมือสร้างผลงานทางวิศวกรรมใด วิศวกรจำเป็นต้องผ่านกระบวนการที่สำคัญอย่างหนึ่ง ก่อนคือขั้นตอนการออกแบบ ในระบบซอฟต์แวร์ก็เช่นเดียวกัน ผู้ที่จะพัฒนาระบบซอฟต์แวร์ที่จะต้องใช้ภาษา ที่มีโปรแกรมเชิงวัตถุก็ควรจะต้องมีการออกแบบระบบซอฟต์แวร์ด้วยกระบวนการพัฒนาเชิงวัตถุเช่นเดียวกัน การออกแบบด้วยวิธีเชิงวัตถุ เป็นการออกแบบกระบวนการในการแยกย่อย และเป็นการใช้สัญลักษณ์แบบการพัฒนา ระบบด้วยวิธีเชิงวัตถุสำหรับ โมเดลระบบทั้งในทางตรรก (Logic) และทางกายภาพ (Physical) รวมถึง โมเดลการทำงานแบบสถิต (Static) และไดนามิก (Dynamic) สำหรับระบบที่ต้องการจะออกแบบ

ด้วยการแยกย่อยปัญหาด้วยวิธีการเชิงวัตถุ ทำให้ออกแบบระบบด้วยวิธีเชิงวัตถุต่างจากการออกแบบ โครงสร้าง เนื่องจากในการออกแบบระบบด้วยวิธีเชิงวัตถุ นั้น ผู้ออกแบบระบบจะมีการแยกแยะเอกลักษณ์โดย ยึดหลักวัตถุเป็นหลัก ส่วนการออกแบบระบบแบบโครงสร้างจะมีการแยกแยะเอกลักษณ์โดยดูจากอัลกอริทึม การทำงานเป็นหลัก

7.5 การวิเคราะห์ระบบด้วยวิธีเชิงวัตถุ (Object-Oriented Analysis : OOA)

การวิเคราะห์ระบบด้วยวิธีเชิงวัตถุ นั้นจะเน้นการโมเดลระบบในโลกความเป็นจริงโดยใช้มุมมองของการ พัฒนาระบบด้วยวิธีเชิงวัตถุ การวิเคราะห์ด้วยวิธีเชิงวัตถุเป็นวิธีวิเคราะห์และตรวจสอบความต้องการ โดยใช้ มุมมองจากคลาสและวัตถุจากคำศัพท์ที่อยู่ในโดเมนของปัญหา

โดยทั้งภาษาการเขียนโปรแกรมเชิงวัตถุและการออกแบบและการวิเคราะห์ระบบด้วยวิธีเชิงวัตถุ นั้น จะเกี่ยวข้องกันดังนี้ คือ หลังจากที่ได้อวิเคราะห์ระบบแล้วก็จะได้โมเดลของระบบเชิงวัตถุ ซึ่งจะใช้เป็นหลักใน การออกแบบ และเมื่อออกแบบเสร็จก็จะได้โครงสร้างหลักหรือพิมพ์เขียว (Blueprint) สำหรับการเขียน โปรแกรมด้วยภาษาในการเขียนโปรแกรมเชิงวัตถุต่อไป

7.6 วัตถุ (Object)

การพัฒนาโปรแกรมโครงสร้างนั้น จะมองระบบเป็นกลุ่มของข้อมูลและฟังก์ชันการทำงานโดยแบ่ง เป็นส่วนย่อยๆ ส่วนการพัฒนา ระบบด้วยมุมมองของการพัฒนาระบบด้วยวิธีเชิงวัตถุจะต่างออกไปก็จะแบ่ง แยกระบบออกเป็นวัตถุ โดยวัตถุในที่นี้คือ

กลุ่มก้อนของ Entity ที่ประกอบด้วยคุณสมบัติหรือส่วนประกอบ (Attribute), พฤติกรรม (Behavior or Method) และอาจจะมีสถานะ (State) ด้วย

แต่ถ้ากล่าวให้ครบถ้วนวัตถุประสงคของวัตถุเป็นสิ่งที่ทั้งข้อมูลและพฤติกรรมวัตถุเป็นสิ่งที่ใช้แทน สิ่งที่มีอยู่ในโลกจริง เช่น ระบบการไหลของอากาศ เครื่องยนต์ หรืออาจจะแทนสิ่งที่มีตัวตนอยู่ในความคิด เช่น บัญชีธนาคาร เครื่องหมายการค้าหรือชื่อนักศึกษา หรือเป็นสิ่งที่มองเห็นได้เช่น ตัวอักษร ดินสอ ปากกา หนังสือ เป็นต้น โดยวัตถุทั้งหมดนั้นเป็นสิ่งที่ผู้พัฒนาสนใจซึ่งประกอบด้วย

- Attribute (ข้อมูล)
- Method (การกระทำ)
- Status (สถานะ)

- Identify (สิ่งบ่งชี้)
- Responsibility (ความรับผิดชอบ)

แนวความคิดของวัตถุคือ การรวบรวมคุณสมบัติที่เกี่ยวข้องไว้ ณ ที่เดียวกัน โดยการพัฒนาแบบแผนโครงสร้างจะออกแบบให้ซอฟต์แวร์มีข้อมูลและฟังก์ชันที่แยกจากกัน

วัตถุนั้นเป็นเครื่องมือในการมองสิ่งต่างๆ ที่มีประสิทธิภาพ เพราะว่าในโลกความเป็นจริงต้องติดต่อกับวัตถุอยู่ตลอดเวลาและวัตถุแต่ละวัตถุมีทั้งส่วนที่สนใจและไม่สนใจ สามารถมีพฤติกรรมหรือไม่มีพฤติกรรมก็ได้ในส่วน of วัตถุที่พฤติกรรมจำเป็นต้องมีการดูแล การโมเดลระบบเชิงวัตถุทำให้สามารถมองระบบต่างๆ ที่ติดต่อกันได้อย่างเข้าใจ ซึ่งการมองระบบแบบโครงสร้างไม่สามารถทำได้

โดยทั่วไปวัตถุอาจจะเป็นสิ่งที่สามารถทำงานได้ด้วยตัวเองซึ่งวัตถุแต่ละตัวจะมีข้อมูล บทบาท พฤติกรรม ที่แตกต่างกันออกไป ซึ่งในแต่ละระบบจะมีความสัมพันธ์กันของแต่ละวัตถุมากขึ้นแตกต่างกันออกไป

7.6.1 แอททริบิวต์ (Attribute)

วัตถุจำเป็นต้องมีการเก็บข้อมูลของตัวเอง โดยข้อมูลที่วัตถุเก็บไว้นั้นอาจจะเก็บไว้เฉยๆ หรือเก็บไว้เพื่อที่จะให้ผู้อื่นเรียกไปใช้งาน เก็บไว้ประมวลผลเพื่อตอบสนองเหตุการณ์ภายนอก หรืออาจจะเก็บไว้เพื่อเป็นสถานะของตัววัตถุเอง สรุปว่าวัตถุส่วนใหญ่จะต้องเก็บข้อมูล เพื่อให้วัตถุนั้นสามารถทำหน้าที่ที่รับผิดชอบได้ถูกต้อง

โดยทั่วไปแอททริบิวต์ของวัตถุจะถูกซ่อนไว้จากผู้ที่ใช้งานวัตถุ ทั้งนี้เนื่องจากการเข้าไปใช้งานแอททริบิวต์โดยตรงเปรียบเสมือนเป็นการเข้าถึงโครงสร้างของวัตถุ ดังนั้นการออกแบบวัตถุที่ดีจึงควรซ่อนแอททริบิวต์ไว้เพื่อไม่ให้ผู้ใช้เห็น

บางครั้งการออกแบบวัตถุที่ไม่ดีก็อาจจะทำให้วัตถุนั้นมีแต่แอททริบิวต์โดยไม่มีเมธอดได้ ซึ่งในกรณีนี้นักออกแบบจะต้องย้อนกลับไปดูว่าวัตถุนั้นควรจะมีเมธอดอะไร เพราะวัตถุที่มีแต่แอททริบิวต์นั้นก็ไม่ได้ต่างจากโครงสร้างข้อมูล

7.6.2 เมธอดหรือพฤติกรรม

วัตถุที่ทำงานแบบพาสลิฟ (Passive Object) จะมีบริการไว้ให้วัตถุอื่น เรียกใช้ ส่วนวัตถุที่ทำงานแบบแอกทีฟ (Active Object) นั้นจะเป็นวัตถุหลักของเซต ซึ่งจะมีหน้าที่เรียกใช้บริการจากวัตถุที่ทำงานแบบพาสลิฟโดยเมธอดแบ่งออกเป็น 3 แบบคือ

1. Simple Method

วัตถุจะให้บริการกับการร้องขอเข้ามาและไม่ค้างค่าในหน่วยความจำไว้สำหรับให้บริการครั้งต่อไป การกระทำเป็นหนึ่งเดียวและสมบูรณ์ในตัว วัตถุพื้นฐานจะมีข้อมูลชนิดพื้นฐานและการทำงานที่เกี่ยวข้องกับข้อมูลเหล่านี้

2. Automation Method

ผู้พัฒนาจะมองวัตถุเป็นเสมือนหนึ่งเป็นเครื่องจักร (State Machine) วัตถุชนิดนี้มีกลุ่มของสถานะที่มีขอบเขต วัตถุหนึ่งอาจจะมีสถานะ 1 หรือมากกว่า ณ เวลาใดๆ การที่วัตถุจะอยู่ในสถานะใดๆ นั้นก็จะขึ้นอยู่กับอีเวนต์ (Event) หรือเหตุการณ์ที่เข้ามาของสภาวะภายนอก เนื่องจากวัตถุชนิดนี้ทำงานแบบ State Machine เพื่อตอบสนองกับเหตุการณ์ภายนอก จึงเรียกวัดูลชนิดนี้ว่า วัตถุแบบรีแอกทีฟ (Reactive Object)

3. Continuous Method

วัตถุจะมีกลุ่มของสถานะไม่จำกัดและไม่มีการขอบเขต การทำงานของวัตถุปัจจุบันจะขึ้นอยู่กับเมธอดและอินพุตก่อนหน้าซึ่งความเกี่ยวข้องระหว่างสถานะนี้อาจจะเป็นความสัมพันธ์แบบต่อเนื่อง ตัวอย่างเช่น ระบบ Fuzzy และตัวควบคุม PID ซึ่งเป็นเครื่องมือสร้างจำนวนแบบ Pseudo-random และตัวกรองดิจิทัล การทำงานของเมธอดของระบบเหล่านี้จะขึ้นอยู่กับอินพุตก่อนหน้า

7.6.3 Message

การสื่อสารกันระหว่างวัตถุสามารถทำได้โดยการส่ง message ซึ่งเป็นการแยกแยะเอกลักษณ์ของข้อมูลหรือข้อสารสนเทศ การควบคุมที่ส่งมาจากวัตถุหนึ่งไปยังอีกวัตถุอื่นๆ ในเชิงการเขียนโปรแกรม message สามารถทำงานได้หลายอย่างเช่น การเรียกฟังก์ชัน อินเตอร์รัพท์ (interrupt) เป็นต้น

ในการวิเคราะห์ระบบจะมีการกำหนด message ส่วนในขั้นตอนของการออกแบบนั้นจึงจะมากำหนดรูปแบบการซิงโครไนเซชัน (synchronization) และความต้องการทางด้านเวลาของแต่ละ message และเมื่อวัตถุได้รับ message มา วัตถุจะเปลี่ยน message ที่เข้ามาเป็นการทำงานเปลี่ยนสถานะ , คำสั่งหรือข้อมูลตามความเหมาะสม

การใช้ message ทำให้แต่ละวัตถุมีความเกี่ยวข้องกันน้อยลง ในขั้นตอนการวิเคราะห์ผู้พัฒนาระบบจะไม่ได้กำหนดรายละเอียดในการติดต่อของ message ว่าจะมีรายละเอียดของการเรียกฟังก์ชัน และเมื่อถึงขั้นตอนการออกแบบจึงจะจัดการกับรายละเอียดเหล่านี้

ส่วนอินเตอร์เฟซ (interface) ของวัตถุเป็นส่วนที่วัตถุใช้ติดต่อกับโลกภายนอก ซึ่งอินเตอร์เฟซจะทำหน้าที่กำหนดชุดของโปรโตคอล (protocol) เพื่อสื่อสารกับวัตถุอื่นๆ โปรโตคอลของส่วนอินเตอร์เฟซนั้นประกอบด้วย 3 ส่วนคือ เงื่อนไขก่อนหน้า (Precondition), ซิกเนเจอร์ (Signature) และ เงื่อนไขท้าย (Postcondition)

precondition เป็นเงื่อนไขที่จะต้องมีความเป็นจริงก่อนที่จะส่งหรือรับ message โดยปกติ precondition จะเป็นหน้าที่ของวัตถุที่ส่ง message จะต้องตรวจสอบ ส่วน postcondition จะต้องเป็นเงื่อนไขที่มีค่าจริงหลังจากวัตถุได้ประมวลผล message เรียบร้อยแล้ว และเป็นหน้าที่ของวัตถุที่รับ message สำหรับ signature นั้น

เป็นรูปแบบในการส่ง message ซึ่งจะเป็นการเรียกฟังก์ชันพร้อมพารามิเตอร์และค่าที่คืนกลับมา (Return Type) หรือ message post/pend ของ RTOS หรือข้อตกลงของ message bus

อินเทอร์เฟซเป็นส่วนที่สำคัญในการแสดงคุณลักษณะของวัตถุ และสิ่งที่อยู่ภายนอกก็จะต้องเห็นส่วนอินเทอร์เฟซของวัตถุด้วย วัตถุสามารถซ่อนรายละเอียดที่ไม่สำคัญสำหรับผู้ใช้ได้หรือเรียกว่า Encapsulation ตัวอย่าง เช่น ภาษา Java ก็มีคำสั่งเพื่อใช้ซ่อนแอตทริบิวต์และเมธอดได้

7.6.4 ความรับผิดชอบ (Responsibility)

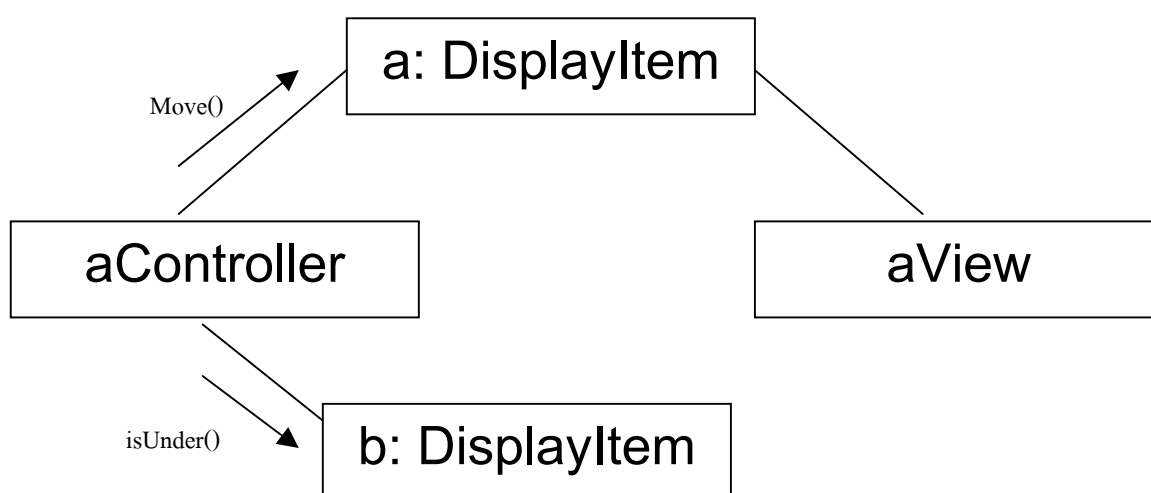
ความรับผิดชอบของวัตถุคือหน้าที่และบทบาทของวัตถุต่อระบบ โดยส่วนอินเทอร์เฟซและเมธอดของวัตถุนั้นสามารถใช้เพื่อให้วัตถุทำหน้าที่รับผิดชอบได้ ความรับผิดชอบของวัตถุควรจะเป็นสิ่งแรกที่คุณพัฒนาระบบกำหนดให้กับวัตถุ เพราะถ้าไม่รู้ว่าวัตถุนั้นมีหน้าที่อย่างไร ก็จะไม่สามารถกำหนดคุณลักษณะอื่นๆ ให้กับวัตถุได้อย่างถูกต้องและครบถ้วน

7.6.5 ความสัมพันธ์ระหว่างวัตถุ

โดยปกติแล้ววัตถุเพียงวัตถุเดียวไม่สามารถที่จะรับผิดชอบงานทั้งหมดของระบบได้วัตถุจะต้องมีความสัมพันธ์และต้องทำงานร่วมกันเพื่อให้ระบบสามารถทำหน้าที่รับผิดชอบได้ โดยเราแบ่งความสัมพันธ์ของวัตถุออกเป็นสองแบบ คือ Link และ Aggregation

1. Link

ความสัมพันธ์ระหว่างวัตถุแบบ Link เป็นการเชื่อมต่อระหว่างวัตถุในทางกายภาพหรือทางตรรก การที่วัตถุซึ่งมาประกอบกันเป็นระบบจะทำงานร่วมกันได้นั้น วัตถุเหล่านี้จะต้องมีการสื่อสารระหว่างกัน วัตถุจะสื่อสารกันโดยวัตถุที่เรียกใช้บริการ (service) จากวัตถุที่เรียกว่า วัตถุไคลเอนต์ (Client) ส่วนวัตถุที่มีบริการให้วัตถุอื่นเรียกว่า วัตถุเซิร์ฟเวอร์ (Server) หรือ Supplier



รูปที่ 7.1 ความสัมพันธ์แบบลิงค์ของวัตถุ

2. Aggregation

Aggregation เป็นความสัมพันธ์ที่วัตถุหนึ่งเป็นส่วนหนึ่งของอีกวัตถุหนึ่ง โดยอาจจะเป็นความสัมพันธ์แบบที่เป็นส่วนหนึ่งแบบธรรมดาหรือคอมโพสิชัน (Composition) ซึ่งมีความแตกต่างกันถ้าเป็นความสัมพันธ์แบบคอมโพสิชันและคลาสที่ประกอบไปด้วยคลาสอื่นไม่สามารถแชร์ความเป็นเจ้าของได้ ส่วนคลาสที่เป็นเจ้าของก็จะมีหน้าที่สร้างและทำลายวัตถุของคลาสที่ประกอบอยู่ภายในด้วย ในการวาดโมเดลจะเขียนคลาสที่เป็นส่วนหนึ่งของคอมโพสิชันโดยใช้สัญลักษณ์เดียวกับ Aggregation ตัวอย่างการใช้ความสัมพันธ์แบบคอมโพสิชันคือ วัตถุทำงานแบบแอคทีฟซึ่งวัตถุที่ทำงานแบบแอคทีฟจะทำหน้าที่สร้างเธรดขึ้นมาเพื่อทำงาน โดยวัตถุที่เป็นส่วนประกอบก็จะทำงานอยู่ในเธรด (thread) ตัววัตถุซึ่งถูกสร้างจากคลาสที่เป็นเจ้าของจะทำหน้าที่รับ message จากภายนอก และส่ง message ที่ได้รับมาต่อให้กับวัตถุที่สร้างคลาสที่เป็นส่วนประกอบ

7.7 คลาส

ในโลกของการพัฒนาระบบด้วยวิธีเชิงวัตถุ คำว่าคลาสคือ การแยกแยะเอกลักษณ์ของคุณสมบัติพื้นฐานจากกลุ่มของวัตถุที่เหมือนกัน อาจกล่าวได้ว่าคลาสเป็นชนิดของวัตถุ ค่า 0 , -3 หรือ 4564 เป็น instance ของคลาส Integer ซึ่งแบ่งคุณสมบัติพื้นฐานที่ใช้ร่วมกันอยู่จะแตกต่างที่ instance แต่ว่าทุก instance จะต้องมีความสัมพันธ์นี้อยู่

คลาสกำหนดแอตทริบิวต์และเมธอดของวัตถุ ซึ่งเป็น instance แต่ว่าคลาสไม่มีความรับผิดชอบคุณสมบัติทั้งหมดของคลาสอยู่ในรูปของชนิด (Type) อย่างไรก็ตามความรับผิดชอบขึ้นอยู่กับการใช้วัตถุในสิ่งแวดล้อม เช่น วัตถุคอนเทนเนอร์ (Container Object) พื้นฐาน อาจเก็บบัญชีของลูกค้าทุกคนส่วนวัตถุคอนเทนเนอร์ของคลาสเดียวกันอาจจะเก็บเรคคอร์ดของคลังสินค้า และยังสามารถเข้าใช้ระบบของสินค้าคงคลังได้อีกด้วย จะเห็นได้ว่าความรับผิดชอบในชนิดนั้นจะเหมือนกันแต่จะแตกต่างกันหากนำไปใช้งานจริง คลาสของวัตถุกำหนดหน้าที่ความรับผิดชอบของวัตถุเฉพาะเมื่อวัตถุที่ถูกสร้างขึ้นจากคลาสทั้งหมดใช้ในสภาวะแวดล้อมและมีหน้าที่เหมือนกัน

7.7.1 ความสัมพันธ์ระหว่างคลาส

1. Association

เมื่อวัตถุหนึ่งใช้บริการของอีกวัตถุหนึ่งแต่ไม่ได้เป็นเจ้าของวัตถุนั้น จะเรียกความสัมพันธ์นี้ว่าแอสโซซิเอชัน (Association) ซึ่งมีความสัมพันธ์แบบนี้จะใช้ได้อย่างเหมาะสมก็ต่อเมื่อ

- วัตถุใช้บริการของอีกวัตถุหนึ่ง แต่ไม่ได้มีความสัมพันธ์แบบเป็นส่วนหนึ่งของ
- วงจรชีวิตของคลาสที่ใช้ไม่ได้เป็นหน้าที่ความรับผิดชอบของคลาสที่เรียกใช้ทั้งการสร้างและทำลายวัตถุ
- มีความสัมพันธ์ระหว่างวัตถุนั้นน้อยกว่าความสัมพันธ์แบบเป็นส่วนหนึ่งของ
- มีความสัมพันธ์ในลักษณะของไคลเอ็นต์-เซิร์ฟเวอร์
- วัตถุมีการถูกเรียกใช้บริการจากหลายๆ วัตถุ และถูกใช้มากเท่าๆ กับที่วัตถุอื่นๆ เรียกใช้

2. Inheritance

เมื่อคลาสหนึ่งเป็นความเฉพาะของอีกคลาสหนึ่งจะใช้ความสัมพันธ์ที่เรียกว่า การสืบทอดคุณสมบัติ หรือ Generalization หรือ Inheritance หมายความว่าคลาสลูกจะมีคุณสมบัติทุกอย่างที่คลาสพ่อแม่มี ถึงแม้ว่าคลาสลูกจะมีความเฉพาะมากกว่าก็ตาม คลาสลูกอาจจะเพิ่มคุณสมบัติของคลาสพ่อแม่โดยการเพิ่มแอตทริบิวต์และเมธอดเข้าไป เรียกความสัมพันธ์นี้ว่า “เป็นชนิดหนึ่งของ” เช่นสัตว์เลี้ยงลูกด้วยนมเป็นชนิดหนึ่งของสัตว์ เป็นต้น

เพื่อความสะดวกต่อการนำกลับมาใช้ใหม่ ผู้พัฒนาสามารถสร้างลำดับชั้นของชนิด (Type Hierarchies) ขึ้นจากคลาส และความสัมพันธ์ในการสืบทอดคุณสมบัติ

3. Aggregation

Aggregation นั้นจะใช้ก็ต่อเมื่อมีวัตถุหนึ่งบรรจุอีกวัตถุหนึ่งไว้ทั้งในทางตรรกและทางกายภาพ คลาสที่ใหญ่กว่าจะเรียกว่าเจ้าของหรือ Owner หรือ Whole ส่วนคลาสที่เล็กกว่านั้นเรียกว่า คลาสส่วนประกอบหรือ คอมโพเนนต์ (Component) หรือ Owned หรือ Part โดยทั่วไป คลาสเจ้าของมีหน้าที่สร้างและทำลายได้หลายคลาส และเมื่อวัตถุมีหลายเจ้าของ ผู้พัฒนาจะพิจารณาว่าคลาสใดจะทำหน้าที่สร้างและทำลายคลาสคอมโพเนนต์นั้น